

```
#property copyright "Copyright 2025, #ea6"
#property link      "https:t.me/BallerDolls"
#property version   "1.00"
#property strict
int OnInit()
{
    OnReInit();
    return(INIT_SUCCEEDED);
}

void OnDeinit(const int reason){}
input int Commission=0;
double com=Commission*Point;
input int StopLoss=0;
double SL=StopLoss*Point;
input int TakeProfit=0;
double TP=TakeProfit*Point;
input double lot=0.01;
input int slip=100;
input int max=60;
input int min=3;
int x=max+2;
input bool Cc = true;
input bool cC = true;
bool invert = true;
bool KC;
static int tag = -1;
static int prime = -1;
static int dime = -1;
static int mem = -1;
int tick = -1;
int y=min-2;
int j;
double signal = 0;
    bool signature = false;
double spread = Ask - Bid;
int FVG=-1;
static string bl="";
double BL[];
double cA[];
double cADX;
double mSO;
double sSO;
double iSO;
double aRVI;
double bRVI;
double cRVI;
double cAC;
double cForce;
double cOBV;
double cAD;
double cMFI;
double cMomentum;
double cDM;
double cWPR;
double cCCI;
double cRSI;
double iA[];
double iATR;
double iStdDev;
double iADX;
double mStochastic;
double sStochastic;
double iStochastic;
double mRVI;
double sRVI;
double iRVI;
double iAC;
double iForce;
double iOBV;
double iAD;
double iMFI;
double iMomentum;
double iDM;
double iWPR;
double iCCI;
double iRSI;
double iHKt;
double iHKk;
double kA[];
double iA[];
double iHKk[];
double iHKt[];
double RSI[];
double CCI[];
double MOM[];
double AD[];
double OBV[];
double Force[];
double MFI[];
double DeM[];
double RVIm[];
double AC[];
double StdDev[];
double ATR[];
double ADX[];
double Supply;
double iSupply;
double Demand;
double iDemand;
double f=100*(2.0/3);
double g=100*(1.0/3);
double gf=100*((2.0/5)/3);
int m;
int n;
static string bottomLine="";
string Regime[];
static double Premium[];
static double Discount[];
static double HH[];
static double LL[];
bool k[];
bool l[];
bool R=true;
bool U[];
double bSL;
double sSL;
double bTP;
double sTP;
int lOrder_id=-1;
int kOrder_id=-1;
int Buy=-1;
int Sell=-1;
bool A=true;
bool B=true;
bool a=true;
bool b=true;
bool ab=false;
bool ba=!ab;
static double D;
static double E;
static double p;
static double q;
bool K=false;
bool c=cC;
bool C=cC;
bool u=false;
bool v=false;
bool iC=Cc;
bool jC=Cc;
static int Z=y+1;
static int z=y+1;
static int O=y+1;
static int o=y+1;
static int r;
static int W=y+1;
static int w=y+1;
static int I;
static int iI;
static int J;
static int iJ;
static int ij;
static int h;
static int count=0;
static int toll=0;
string tally="";
bool tickTock=false;
//Open[2]
double iopen;
//Close[2]
double iPrice;
static int iZ=y+1;
static int iz=y+1;
static int iW=y+1;
static int iw=y+1;
static int iO=y+1;
static int io=y+1;
static int ir;
```



```
int S=x;
int T=x;
int X=y;
int Y=y;
bool FG=false;
bool GF=false;
double price;
double Price;
double open;
double iH;
double iL;
double Sale;
double iSale;
double Stock;
double iStock;
static datetime t;
void OnReInit()
{
    KC = invert;
    ArrayInitialize(cA, 0); //ArrayResize(cA, 0);
    ArrayInitialize(iA, 0); //ArrayResize(iA, 0);
    ArrayInitialize(kA, 0); //ArrayResize(kA, 0);
    ArrayInitialize(lA, 0); //ArrayResize(lA, 0);
    ArrayInitialize(IHKK, 0); //ArrayResize(IHKK, 0);
    ArrayInitialize(IHKt, 0); //ArrayResize(IHKt, 0);
    ArrayInitialize(RSI, 0); //ArrayResize(RSI, 0);
    ArrayInitialize(CCI, 0); //ArrayResize(CCI, 0);
    ArrayInitialize(MOM, 0); //ArrayResize(MOM, 0);
    ArrayInitialize(AD, 0); //ArrayResize(AD, 0);
    ArrayInitialize(OBV, 0); //ArrayResize(OBV, 0);
    ArrayInitialize(Force, 0); //ArrayResize(Force, 0);
    ArrayInitialize(MFI, 0); //ArrayResize(MFI, 0);
    ArrayInitialize(DeM, 0); //ArrayResize(DeM, 0);
    ArrayInitialize(RVIm, 0); //ArrayResize(RVIm, 0);
    ArrayInitialize(AC, 0); //ArrayResize(AC, 0);
    ArrayInitialize(StdDev, 0); //ArrayResize(StdDev, 0);
    ArrayInitialize(ATR, 0); //ArrayResize(ATR, 0);
    ArrayInitialize(ADX, 0); //ArrayResize(ADX, 0);
    ArrayInitialize(StdDev, 0); //ArrayResize(StdDev, 0);
    /*ArrayInitialize(Regime, "");*/ArrayResize(Regime, 0);
    ArrayInitialize(Premium, 0); //ArrayResize(Premium, 0);
    ArrayInitialize(Discount, 0); //ArrayResize(Discount, 0);
    ArrayInitialize(HH, 0); //ArrayResize(HH, 0);
    ArrayInitialize(LL, 0); //ArrayResize(LL, 0);
    ArrayInitialize(k, 0); //ArrayResize(k, 0);
    ArrayInitialize(l, 0); //ArrayResize(l, 0);
    ArrayInitialize(U, 0); //ArrayResize(U, 0);
    R=true;
    //    ab=false;
    D=0;
    E=0;
    K=false;
    Z=y+1;
    z=y+1;
    O=y+1;
    o=y+1;
    r=0;
    W=y+1;
    w=y+1;
    I=0;
    iI=0;
    J=0;
    iJ=0;
    ij=0;
    //    h=0;
    toll=0;
    tally="";
    tickTock=false;
    iZ=y+1;
    iz=y+1;
    iW=y+1;
    iw=y+1;
    iO=y+1;
    io=y+1;
    ir=0;
    S=x;
    T=x;
    X=y;
    Y=y;
    FG=false;
    Print("ReSet");
}
void Unify()
{
    ArrayResize(ATR, j+1);
    for(int i=0; i<j+1; i++){ATR[i]=iATR(NULL, 0, j, i);}
    double maxATR=ATR[ArrayMaximum(ATR, WHOLE_ARRAY, 0)];
    double minATR=ATR[ArrayMinimum(ATR, WHOLE_ARRAY, 0)];
    double rangeATR=maxATR-minATR;
    if(rangeATR!=0) iATR=100*((iATR(NULL, 0, j, 0)-minATR)/rangeATR);
    ArrayResize(StdDev, j+1);
    for(int i=0; i<j+1; i++){StdDev[i]=iStdDev(NULL, 0, j, 0, MODE_SMA, PRICE_CLOSE, i);}
    double maxSD=StdDev[ArrayMaximum(StdDev, WHOLE_ARRAY, 0)];
    double minSD=StdDev[ArrayMinimum(StdDev, WHOLE_ARRAY, 0)];
    double rangeSD=maxSD-minSD;
    if(rangeSD!=0) iStdDev=100*((iStdDev(NULL, 0, j, 0, MODE_SMA, PRICE_CLOSE, 0)-minSD)/rangeSD);
}
void Normalize()
{
    iSupply=iBands(NULL, 0, j, 2, 0, PRICE_CLOSE, MODE_UPPER, 0);
    iSupply=iBands(NULL, 0, j, 2, 0, PRICE_CLOSE, MODE_UPPER, 1);
    iDemand=iBands(NULL, 0, j, 2, 0, PRICE_CLOSE, MODE_LOWER, 0);
    iDemand=iBands(NULL, 0, j, 2, 0, PRICE_CLOSE, MODE_LOWER, 1);
    ArrayResize(iA, 13*((S+1)-Y));
    ArrayResize(cA, 13*((S+1)-Y));
    double uADX[];
    ArrayResize(uADX, j+1);
    for(int i=0; i<j+1; i++){uADX[i]=iADX(NULL, 0, j, PRICE_CLOSE, MODE_PLUSDI, i);}
    double maxuADX=uADX[ArrayMaximum(uADX, WHOLE_ARRAY, 0)];
    double minuADX=uADX[ArrayMinimum(uADX, WHOLE_ARRAY, 0)];
    double lADX[];
    ArrayResize(lADX, j+1);
    for(int i=0; i<j+1; i++){lADX[i]=iADX(NULL, 0, j, PRICE_CLOSE, MODE_MINUSDI, i);}
    double maxlADX=lADX[ArrayMaximum(lADX, WHOLE_ARRAY, 0)];
    double minlADX=lADX[ArrayMinimum(lADX, WHOLE_ARRAY, 0)];
    ArrayResize(ADX, j+1);
    for(int i=0; i<j+1; i++){ADX[i]=iADX(NULL, 0, j, PRICE_CLOSE, MODE_MAIN, i);}
    double maxmADX=ADX[ArrayMaximum(ADX, WHOLE_ARRAY, 0)];
    double minmADX=ADX[ArrayMinimum(ADX, WHOLE_ARRAY, 0)];
    double maxADX=MathMax(maxmADX, MathMax(maxuADX, maxlADX));
    double minADX=MathMin(minmADX, MathMin(minuADX, minlADX));
    double rangeADX=maxADX-minADX;
    if(rangeADX!=0)
    {
        iADX=MathAbs(100*((iADX(NULL, 0, j, PRICE_CLOSE, MODE_MAIN, 0)-minADX)/rangeADX));
        iA[0*(S-Y)+(j-(Y+1))]=iADX;
        cADX=MathAbs(100*((ADX[1]-minADX)/rangeADX));
        cA[0*(S-Y)+(j-(Y+1))]=cADX;
    }
    int jS0=(int)MathRound((double)j*3.0/5);
    mStochastic=iStochastic(NULL, 0, j, 3, jS0, MODE_SMA, 0, MODE_MAIN, 0);
    sStochastic=iStochastic(NULL, 0, j, 3, jS0, MODE_SMA, 0, MODE_SIGNAL, 0);
    iStochastic=(mStochastic+sStochastic)/2;
    iA[1*(S-Y)+(j-(Y+1))]=iStochastic;
    mS0=iStochastic(NULL, 0, j, 3, jS0, MODE_SMA, 0, MODE_MAIN, 1);
    sS0=iStochastic(NULL, 0, j, 3, jS0, MODE_SMA, 0, MODE_SIGNAL, 1);
    iS0=(mS0+sS0)/2;
    cA[1*(S-Y)+(j-(Y+1))]=iS0;
    ArrayResize(RVIm, j+1);
    for(int i=0; i<j+1; i++){RVIm[i]=iRVI(NULL, 0, j, MODE_MAIN, i);}
    double maxMRVI=RVIm[ArrayMaximum(RVIm, WHOLE_ARRAY, 0)];
    double minMRVI=RVIm[ArrayMinimum(RVIm, WHOLE_ARRAY, 0)];
    double RVIs[];
    ArrayResize(RVIs, j+1);
    for(int i=0; i<j+1; i++){RVIs[i]=iRVI(NULL, 0, j, MODE_SIGNAL, i);}
    double maxSRVI=RVIs[ArrayMaximum(RVIs, WHOLE_ARRAY, 0)];
    double minSRVI=RVIs[ArrayMinimum(RVIs, WHOLE_ARRAY, 0)];
    double maxRVI=MathMax(maxMRVI, maxSRVI);
    double minRVI=MathMin(minMRVI, minSRVI);
    double rangeRVI=maxRVI-minRVI;
    if(rangeRVI!=0)
    {
        mRVI=100*((iRVI(NULL, 0, j, MODE_MAIN, 0)-minRVI)/rangeRVI);
        sRVI=100*((iRVI(NULL, 0, j, MODE_SIGNAL, 0)-minRVI)/rangeRVI);
        iRVI=(mRVI+sRVI)/2;
        iA[2*(S-Y)+(j-(Y+1))]=iRVI;
        aRVI=100*((iRVI(NULL, 0, j, MODE_MAIN, 1)-minRVI)/rangeRVI);
        bRVI=100*((iRVI(NULL, 0, j, MODE_SIGNAL, 1)-minRVI)/rangeRVI);
        cRVI=(aRVI+bRVI)/2;
        cA[2*(S-Y)+(j-(Y+1))]=cRVI;
    }
    ArrayResize(AC, j+1);
    for(int i=0; i<j+1; i++){AC[i]=iAC(NULL, 0, i);}
    double maxAC=AC[ArrayMaximum(AC, WHOLE_ARRAY, 0)];
    double minAC=AC[ArrayMinimum(AC, WHOLE_ARRAY, 0)];
    double rangeAC=maxAC-minAC;
    if(rangeAC!=0)
    {
        iAC=MathAbs(100*((iAC(NULL, 0, 0)-minAC)/rangeAC));
        iA[3*(S-Y)+(j-(Y+1))]=iAC;
        cAC=MathAbs(100*((iAC(NULL, 0, 1)-minAC)/rangeAC));
    }
```



```

cA[3*(S-Y)+(j-(Y+1))]=cAC;
}
ArrayResize(Force,j+1);
for(int i=0;i<j+1; i++){Force[i]=iForce(NULL,0,j,MODE_SMA,PRICE_CLOSE,i);}
double maxForce=Force[ArrayMaximum(Force,WHOLE_ARRAY,0)];
double minForce=Force[ArrayMinimum(Force,WHOLE_ARRAY,0)];
double rangeForce=maxForce-minForce;
if(rangeForce!=0)
{
    iForce=100*((iForce(NULL,0,j,MODE_SMA,PRICE_CLOSE,0)-minForce)/rangeForce);
    iA[4*(S-Y)+(j-(Y+1))]=iForce;
    cForce=100*((iForce(NULL,0,j,MODE_SMA,PRICE_CLOSE,1)-minForce)/rangeForce);
    cA[4*(S-Y)+(j-(Y+1))]=cForce;
}
ArrayResize(OBV,j+1);
for(int i=0;i<j+1; i++){OBV[i]=iOBV(NULL,0,PRICE_CLOSE,i);}
double maxOBV=OBV[ArrayMaximum(OBV,WHOLE_ARRAY,0)];
double minOBV=OBV[ArrayMinimum(OBV,WHOLE_ARRAY,0)];
double rangeOBV=maxOBV-minOBV;
if(rangeOBV!=0)
{
    iOBV=100*((iOBV(NULL,0,PRICE_CLOSE,0)-minOBV)/rangeOBV);
    iA[5*(S-Y)+(j-(Y+1))]=iOBV;
    cOBV=100*((iOBV(NULL,0,PRICE_CLOSE,1)-minOBV)/rangeOBV);
    cA[5*(S-Y)+(j-(Y+1))]=cOBV;
}
ArrayResize(AD,j+1);
for(int i=0;i<j+1; i++){AD[i]=iAD(NULL,0,i);}
double maxAD=AD[ArrayMaximum(AD,WHOLE_ARRAY,0)];
double minAD=AD[ArrayMinimum(AD,WHOLE_ARRAY,0)];
double rangeAD=maxAD-minAD;
if(rangeAD!=0)
{
    iAD=100*((iAD(NULL,0,0)-minAD)/rangeAD);
    iA[6*(S-Y)+(j-(Y+1))]=iAD;
    cAD=100*((iAD(NULL,0,1)-minAD)/rangeAD);
    cA[6*(S-Y)+(j-(Y+1))]=cAD;
}
ArrayResize(MFI,j+1);
for(int i=0;i<j+1; i++){MFI[i]=iMFI(NULL,0,j,i);}
double maxMFI=MFI[ArrayMaximum(MFI,WHOLE_ARRAY,0)];
double minMFI=MFI[ArrayMinimum(MFI,WHOLE_ARRAY,0)];
double rangeMFI=maxMFI-minMFI;
if(rangeMFI!=0)
{
    iMFI=100*((iMFI(NULL,0,j,0)-minMFI)/rangeMFI);
    iA[7*(S-Y)+(j-(Y+1))]=iMFI;
    cMFI=100*((iMFI(NULL,0,j,1)-minMFI)/rangeMFI);
    cA[7*(S-Y)+(j-(Y+1))]=cMFI;
}
ArrayResize(MOM,j+1);
for(int i=0;i<j+1; i++){MOM[i]=iMomentum(NULL,0,j,PRICE_CLOSE,i);}
double maxMOM=MOM[ArrayMaximum(MOM,WHOLE_ARRAY,0)];
double minMOM=MOM[ArrayMinimum(MOM,WHOLE_ARRAY,0)];
double rangeMOM=maxMOM-minMOM;
if(rangeMOM!=0)
{
    iMomentum=100*((iMomentum(NULL,0,j,PRICE_CLOSE,0)-minMOM)/rangeMOM);
    iA[8*(S-Y)+(j-(Y+1))]=iMomentum;
    cMomentum=100*((iMomentum(NULL,0,j,PRICE_CLOSE,1)-minMOM)/rangeMOM);
    cA[8*(S-Y)+(j-(Y+1))]=cMomentum;
}
ArrayResize(DeM,j+1);
for(int i=0;i<j+1; i++){DeM[i]=iDeMarker(NULL,0,j,i);}
double maxDM=DeM[ArrayMaximum(DeM,WHOLE_ARRAY,0)];
double minDM=DeM[ArrayMinimum(DeM,WHOLE_ARRAY,0)];
double rangeDM=maxDM-minDM;
if(rangeDM!=0)
{
    iDM=100*((iDeMarker(NULL,0,j,0)-minDM)/rangeDM);
    iA[9*(S-Y)+(j-(Y+1))]=iDM;
    cDM=100*((iDeMarker(NULL,0,j,1)-minDM)/rangeDM);
    cA[9*(S-Y)+(j-(Y+1))]=cDM;
}
iWPR=iWPR(NULL,0,j,0)+100;
iA[10*(S-Y)+(j-(Y+1))]=iWPR;
cWPR=iWPR(NULL,0,j,1)+100;
cA[10*(S-Y)+(j-(Y+1))]=cWPR;
ArrayResize(CCI,j+1);
for(int i=0;i<j+1; i++){CCI[i]=iCCI(Symbol(),0,j,PRICE_TYPICAL,i);}
double maxCCI=CCI[ArrayMaximum(CCI,WHOLE_ARRAY,0)];
double minCCI=CCI[ArrayMinimum(CCI,WHOLE_ARRAY,0)];
double rangeCCI=maxCCI-minCCI;
if(rangeCCI!=0)
{
    iCCI=100*((iCCI(Symbol(),0,j,PRICE_TYPICAL,0)-minCCI)/rangeCCI);
    iA[11*(S-Y)+(j-(Y+1))]=iCCI;
    cCCI=100*((iCCI(Symbol(),0,j,PRICE_TYPICAL,1)-minCCI)/rangeCCI);
    cA[11*(S-Y)+(j-(Y+1))]=cCCI;
}
ArrayResize(RSI,j+1);
for(int i=0;i<j+1; i++){RSI[i]=iRSI(NULL,0,j,PRICE_CLOSE,i);}
double maxRSI=RSI[ArrayMaximum(RSI,WHOLE_ARRAY,0)];
double minRSI=RSI[ArrayMinimum(RSI,WHOLE_ARRAY,0)];
double rangeRSI=maxRSI-minRSI;
if(rangeRSI!=0)
{
    iRSI=100*((iRSI(NULL,0,j,PRICE_CLOSE,0)-minRSI)/rangeRSI);
    iA[12*(S-Y)+(j-(Y+1))]=iRSI;
    cRSI=100*((iRSI(NULL,0,j,PRICE_CLOSE,1)-minRSI)/rangeRSI);
    cA[12*(S-Y)+(j-(Y+1))]=cRSI;
}
int kIHK=(int)MathRound(((double)j)/2);
int tIHK=(int)MathRound(((double)kIHK+1)/3);
double IHKa[];
double IHKb[];
double IHKc[];
ArrayResize(IHKa,j+1);
for(int i=0;i<j+1; i++){IHKa[i]=iIchimoku(NULL,0,tIHK,kIHK,j,MODE_SENKOUSPANA,i);}
double maxIHKa=IHKa[ArrayMaximum(IHKa,WHOLE_ARRAY,0)];
double minIHKa=IHKa[ArrayMinimum(IHKa,WHOLE_ARRAY,0)];
ArrayResize(IHKb,j+1);
for(int i=0;i<j+1; i++){IHKb[i]=iIchimoku(NULL,0,tIHK,kIHK,j,MODE_SENKOUSPANB,i); }
double maxIHKb=IHKb[ArrayMaximum(IHKb,WHOLE_ARRAY,0)];
double minIHKb=IHKb[ArrayMinimum(IHKb,WHOLE_ARRAY,0)];
ArrayResize(IHKc,j+1);
for(int i=0;i<j+1; i++){IHKc[i]=iIchimoku(NULL,0,tIHK,kIHK,j,MODE_CHIKOUSPAN,26+i);}
double maxIHKc=IHKc[ArrayMaximum(IHKc,WHOLE_ARRAY,0)];
double minIHKc=IHKc[ArrayMinimum(IHKc,WHOLE_ARRAY,0)];
ArrayResize(IHKt,j+1);
for(int i=0;i<j+1; i++){IHKt[i]=iIchimoku(NULL,0,tIHK,kIHK,j,MODE_TENKANSEN,i);}
double maxIHKt=IHKt[ArrayMaximum(IHKt,WHOLE_ARRAY,0)];
double minIHKt=IHKt[ArrayMinimum(IHKt,WHOLE_ARRAY,0)];
ArrayResize(IHKk,j+1);
for(int i=0;i<j+1; i++){IHKk[i]=iIchimoku(NULL,0,tIHK,kIHK,j,MODE_KIJUNSEN,i);}
double maxIHKk=IHKk[ArrayMaximum(IHKk,WHOLE_ARRAY,0)];
double minIHKk=IHKk[ArrayMinimum(IHKk,WHOLE_ARRAY,0)];
double maxIHK=MathMax(maxIHKa,MathMax(maxIHKb,MathMax(maxIHKc,MathMax(maxIHKk,maxIHKt))));
double minIHK=MathMin(minIHKa,MathMin(minIHKb,MathMin(minIHKc,MathMin(minIHKk,minIHKt))));
double rangeIHK=maxIHK-minIHK;
if(rangeIHK!=0)
{
    iIHKk=100*((iIchimoku(NULL,0,tIHK,kIHK,j,MODE_KIJUNSEN,0)-minIHK)/rangeIHK);
    iIHKt=100*((iIchimoku(NULL,0,tIHK,kIHK,j,MODE_TENKANSEN,0)-minIHK)/rangeIHK);
}
}
void M()
{for(int i=0;i<t13; i++)
{
    if(Price>HH[j-(y+1)]) if(((iA[i*(S-Y)+(j-(Y+1))]>f+gf)||cA[i*(S-Y)+(j-(Y+1))]<kA[i*(S-Y)+(j-(Y+1))]]) m++;
    else if(price>HH[j-(y+1)]) if(((iA[i*(S-Y)+(j-(Y+1))]>f+gf)||iA[i*(S-Y)+(j-(Y+1))]<kA[i*(S-Y)+(j-(Y+1))]]) m++;
    else if(iA[i*(S-Y)+(j-(Y+1))]>f+gf) m++;
}
if((iA[0*(S-Y)+(j-(Y+1))]>f+gf)||iA[0*(S-Y)+(j-(Y+1))]<g-gf)) m++;
if((iIHKt>f+gf)&&(iIHKk>f+gf)) m++;
if(Price>HH[j-(y+1)])
{
    ArrayResize(kA,13*(S-Y));
    for(int i=0;i<t13; i++){kA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];}
    HH[j-(y+1)]=Price;
}
}
}
void N()
{for(int i=0;i<t13; i++)
{
    if(Price<LL[j-(y+1)]) if(((iA[i*(S-Y)+(j-(Y+1))]<g-gf)||cA[i*(S-Y)+(j-(Y+1))]>lA[i*(S-Y)+(j-(Y+1))]]) n++;
    else if(price<LL[j-(y+1)]) if(((iA[i*(S-Y)+(j-(Y+1))]<g-gf)||iA[i*(S-Y)+(j-(Y+1))]>lA[i*(S-Y)+(j-(Y+1))]]) n++;
    else if(iA[i*(S-Y)+(j-(Y+1))]<g-gf) n++;
}
if((iA[0*(S-Y)+(j-(Y+1))]>f+gf)||iA[0*(S-Y)+(j-(Y+1))]<g-gf)) n++;
if((iIHKt<g-gf)&&(iIHKk<g-gf)) n++;
if(Price<LL[j-(y+1)])
{
    ArrayResize(lA,13*(S-Y));
    for(int i=0;i<t13; i++){lA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];}
    LL[j-(y+1)]=Price;
}
}
}
```



```

void F()
{
    Normalize();
    //if(j==h) ab=false;
    k[j-(y+1)]=false;
    l[j-(y+1)]=false;
    HH[j-(y+1)]=iH;
    LL[j-(y+1)]=iL;
    Premium[j-(y+1)]=iH;
    Discount[j-(y+1)]=iL;
    ArrayResize(kA,13*(S-Y));
    ArrayResize(lA,13*(S-Y));
    for(int i=0;i<13; i++)
    {
        kA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];
        lA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];
    }
    if((R==true)&&(FG==true))
    {
        ArrayResize(U,x-y);
        int V=0; U[j-(y+1)]=true;
        for(int i=y+1;i<x; i++){if(U[i-(y+1)]==true) V++;}
        if(V==x-y){R=false;} V=0;
    }
}

void G()
{
    double H=iHigh(Symbol(), Period(), 1);
    double L=iLow(Symbol(), Period(), 1);
    ArrayResize(kA,13*(S-Y));
    ArrayResize(lA,13*(S-Y));
    for(j=2;j<h+1; j++)
    {
        if(j==x) break;
        k[j-(y+1)]=false;
        l[j-(y+1)]=false;
        HH[j-(y+1)]=H;
        LL[j-(y+1)]=L;
        Premium[j-(y+1)]=H;
        Discount[j-(y+1)]=L;
        for(int i=0;i<13; i++)
        {
            kA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];
            lA[i*(S-Y)+(j-(Y+1))]=cA[i*(S-Y)+(j-(Y+1))];
        }
    }
}

void S()
{
    if(SL!=0){sSL=Bid+SL-com; bSL=Ask-SL+com;}
    if(TP!=0){sTP=Bid-TP; bTP=Ask+TP;}
}

void T()
{
    if(((b==false)&&(lOrder_id!=-1))||((a==false)&&(kOrder_id!=-1)))
    {
        Buy=lOrder_id; Sell=kOrder_id;
    }
    else if(((b==false)&&(kOrder_id!=-1))||((a==false)&&(lOrder_id!=-1)))
    {
        Buy=kOrder_id; Sell=lOrder_id;
    }
    if(Buy!=-1)
    {
        if(OrderSelect(Buy,SELECT_BY_TICKET))
        {
            /*E=OrderOpenPrice();*/ E=price; q=E+3*com;
        }
    }
    else if(Sell!=-1)
    {
        if(OrderSelect(Sell,SELECT_BY_TICKET))
        {
            /*D=OrderOpenPrice();*/ D=price; p=D-3*com;
        }
    }
    if((K==false)&&((SL!=0)||(com!=0)))
    {
        if((b==false)&&(Price>q))
        {
            b=OrderModify(Buy,E+com,bTP,0,CLR_NONE); K=true;
        }
        else if((a==false)&&(Price<p))
        {
            a=OrderModify(Sell,D-com,sTP,0,CLR_NONE); K=true;
        }
    }
    if((E!=0)&&(price>E/*+com*/)) B=true;
    else if((E!=0)&&(price<E/*+com*/)) B=false;
    if((D!=0)&&(price<D/*-com*/)) A=true;
    else if((D!=0)&&(price>D/*-com*/)) A=false;
}

void Top()
{
    bottomLine=DoubleToString(price, Digits);
    if(ObjectFind(0, bottomLine)==-1)
    {
        ArrayResize(BL, (FVG+1)+1);
        BL[FVG+1]=price;
        ObjectCreate(0, bottomLine, OBJ_HLINE, 0, 0, price);
        ObjectSetInteger(0, bottomLine, OBJPROP_COLOR, clrBlue);
        ObjectSetInteger(0, bottomLine, OBJPROP_STYLE, STYLE_SOLID);
        ObjectSetInteger(0, bottomLine, OBJPROP_WIDTH, 1);
        FVG++; bL=bottomLine;
    }
}

void Bott()
{
    bottomLine=DoubleToString(price, Digits);
    if(ObjectFind(0, bottomLine)==-1)
    {
        ArrayResize(BL, FVG+2);
        BL[FVG+1]=price;
        ObjectCreate(0, bottomLine, OBJ_HLINE, 0, 0, price);
        ObjectSetInteger(0, bottomLine, OBJPROP_COLOR, clrRed);
        ObjectSetInteger(0, bottomLine, OBJPROP_STYLE, STYLE_SOLID);
        ObjectSetInteger(0, bottomLine, OBJPROP_WIDTH, 1);
        FVG++; bL=bottomLine;
    }
}

void Deleter(string obj, double &prices[], int index)
{
    int size = ArraySize(prices);
    if((index<0)||(index>=size)||(FVG!=size-1))
        return; // iInvalid
    if(ObjectFind(0, obj)!=-1)
    {
        ObjectDelete(0, obj); FVG--;
    }
    for(int i=index; i<size-1; i++)
    {
        prices[i]=prices[i+1];
    }
    ArrayResize(prices, size-1);
}

void A()
{
    if((v==true)&&(lOrder_id!=-1))
    {
        int bTrade=OrderClose(lOrder_id,lot,Bid,slip,Blue);
        lOrder_id=-1;
    }
    else if((v==true)&&(kOrder_id!=-1))
    {
        int bTrade=OrderClose(kOrder_id,lot,Bid,slip,Blue);
        kOrder_id=-1;
    }
    E=0; A=true; B=false; K=false; Buy=-1;
}

void B()
{
    if((u==true)&&(kOrder_id!=-1))
    {
        int sTrade=OrderClose(kOrder_id,lot,Ask,slip,Red);
        kOrder_id=-1;
    }
    else if((u==true)&&(lOrder_id!=-1))
    {
        int sTrade=OrderClose(lOrder_id,lot,Ask,slip,Red);
        lOrder_id=-1;
    }
    D=0; A=false; B=true; K=false; Sell=-1;
}

void P()
{
    {S();
    if(signature==true)
    {if(C==true)
    {
        lOrder_id=OrderSend(_Symbol,OP_BUY,lot,Ask,slip,bSL,bTP,"EA",1992470,0,Blue);
        b=false; u=false; v=true;
    }
    else
    {
        lOrder_id=OrderSend(_Symbol,OP_SELL,lot,Bid,slip,sSL,sTP,"EA",1992470,0,Red);
        a=false; u=true; v=false;
    }
    }
}
}

void Q()

```



```
{S();
if(signature==true)
{if(C==true)
{
kOrder_id=OrderSend(_Symbol,OP_SELL,lot,Bid,slip,sSL,STP,"EA",1992470,0,Red);
a=false; u=true; v=false;
}
else
{
kOrder_id=OrderSend(_Symbol,OP_BUY,lot,Ask,slip,bSL,bTP,"EA",1992470,0,Blue);
b=false; u=false; v=true;
}
}
}
}
void H(){M(); if(m>=12) k[j-(y+1)]=true; else{k[j-(y+1)]=false;} m=0;}
void L(){N(); if(n>=12) l[j-(y+1)]=true; else{l[j-(y+1)]=false;} n=0;}
void J()
{if(I==iZ){J=iW;}
else if(I==iW){J=iZ;}
if(iI==iz) iJ=iw;
else if(iI==iw) iJ=iz;
}
void O(int inp,int inp0,int inp1,bool inp2,bool inp3)
{if((inp<inp1)&&((Regime[inp0-(y+1)]=="sRange"))|((Regime[inp0-(y+1)]=="tRange"))){inp2=inp3;}
else if((Regime[inp0-(y+1)]!="sRange")&&(Regime[inp0-(y+1)]!="tRange")) inp2=!inp3; else inp2=!inp3;
}
}
void R()
{if(j<=J){int i=j; O=i; iO=i;}
if((j>J)&&(j<r)){int i=j; O=i; iO=i; r=i;}
else if(j>J){int i=j; r=i;}
if(j<=iJ){int i=j; o=i; iO=i;}
if((j>iJ)&&(j<ir)){int i=j; o=i; iO=i; ir=i;}
else if(j>iJ){int i=j; ir=i;}
}
}
void KC()
{if((E!=0)&&((A==false)&&(B==false))&&(v==true)&&(signal<E)){invert=!KC;}
if((D!=0)&&((B==false)&&(A==false))&&(u==true)&&(signal>D)){invert=!KC;}
}
}
bool OnHold(int inp,string inp0,string inp1){return ((Regime[inp-(y+1)]==inp0)|((Regime[inp-(y+1)]==inp1)));}
bool OnFire(int inp,string inp0,string inp1){return ((Regime[inp-(y+1)]!=inp0)&&(Regime[inp-(y+1)]!=inp1)));}
void OnPoint()
{for(j=y+1;j<x; j++)
{
Unify(); Normalize();
if(((iStdDev<50)&&(iATR>50)) if(Regime[j-(y+1)]!="Stable"){H(); L(); if(OnFire(j,"sVolatile","tVolatile")) Regime[j-(y+1)]="sVolatile";}
else if((iStdDev<50)&&(iATR<50))
{
if(Regime[j-(y+1)]!="Stable")
{
R(); H(); L(); if(OnFire(j,"sRange","tRange")) Regime[j-(y+1)]="sRange";
}
}
else if(OnFire(j,"sTrend","tTrend")) Regime[j-(y+1)]="sTrend";
}
}
}
void Signal(){
ab=!ba; count=0; toll=0; tally=""; signal=price;
}
}
bool OnGaurd(int inp){
if((price>E)&&(E!=0)){if((signature==true)&&(inp!=-1)){dime=0; return true;} else{if((signature==true)&&(inp!=-1)){dime=1; return false;}
if((price<D)&&(D!=0)){if((signature==true)&&(inp!=-1)){dime=1; return true;} else{if((signature==true)&&(inp!=-1)){dime=0; return false;}}
void OnLog(){
if(dime==0){Print("Flying");} else if(dime==1){Print("Falling");} else{Print("Wait/Decern");}
}
}
void OnCall()
{for(j=y+1;j<X+2; j++)
{
Unify(); Normalize();
if((Supply<=price)|(|iSupply<=price)|(|iSupply<=iH))
{
int i=j; I=iW; iZ=i; Z=i; iC=C;
if((iwl=0)&&(jC==Cc)){h=i;} jC=!C;
if(OnHold(j,"sTrend","tTrend")){iz=i; z=i; iI=iw; H();}
if(X!=x-1) X++;
}
if((Demand>=price)|(|iDemand>=price)|(|iDemand>=iL))
{
int i=j; I=iZ; iW=i; W=i; jC=C;
if((izl=0)&&(iC==Cc)){h=i;} iC=!C;
if(OnHold(j,"sTrend","tTrend")){iw=i; w=i; iI=iz; L();}
if(X!=x-1) X++;
}
} X=y;
}
}
void OnBar()
{for(j=y+1;j<x; j++)
{
Unify(); Normalize();
if(((iStdDev<50)&&(iATR>50))
{
if(Regime[j-(y+1)]!="Stable")
{
if(Regime[j-(y+1)]!="tVolatile")
{
F(); H(); L(); Regime[j-(y+1)]="tVolatile";
}
}
}
else if((iStdDev<50)&&(iATR<50))
{
if(Regime[j-(y+1)]!="Stable")
{
R(); H(); L();
if(Regime[j-(y+1)]!="tRange")
{
F(); Regime[j-(y+1)]="tRange";
}
}
}
else if((Regime[j-(y+1)]!="tTrend")&&(Regime[j-(y+1)]!="sTrend")&&(LL[j-(y+1)]<Discount[j-(y+1)]&&(HH[j-(y+1)]>Premium[j-(y+1)])) Regime[j-(y+1)]="Stable";
else
{
if(Regime[j-(y+1)]!="tTrend")
{
F(); Regime[j-(y+1)]="tTrend";
}
}
}
}
}
Stock=iBands(NULL,0,y,2,0,PRICE_CLOSE,MODE_UPPER,0);
Sale=iBands(NULL,0,y,2,0,PRICE_CLOSE,MODE_LOWER,0);
iStock=iBands(NULL,0,y,2,0,PRICE_CLOSE,MODE_UPPER,1);
iSale=iBands(NULL,0,y,2,0,PRICE_CLOSE,MODE_LOWER,1);
iOpen=iOpen(Symbol(),0,2);
iPrice=iClose(Symbol(),0,2);
double lPass=0;
if(Price<open){lPass=Price;} else{lPass=open;}
double ilPass=0;
if(iPrice<iopen){ilPass=iPrice;} else{ilPass=iopen;}
double kPass=0;
if(Price>open){kPass=Price;} else{kPass=open;}
double ikPass=0;
if(iPrice>iopen){kPass=iPrice;} else{kPass=iopen;}
string fg="";
if(signal!=0)
{if(*(price==signal+spread)|(|*(price==signal+com))
{Alert("Long: ",price);
if((count==0)&&(tally=="Sell")){count ++; tally="Buy";}
}
if(*(price<signal-spread)|(|*(price<=signal-com))
{Alert("Short: ",price);
if((count==0)&&(tally=="Buy")){count ++; tally="Sell";}
}
if(tickTock==false)
{if((((Price>signal)|(|(Price>ikPass)|(|(price<kPass)))&&(((iC==Cc)&&(Price>HH[min-(y+1)]))|(|(jC==Cc)&&(Price>=LL[min-(y+1)]&&(((open>=Stock)|(|(Price==Stock))|(|(open>=iStock)|(|(Price>=iStock))))))
{Alert("Buy: ",price);
if((toll==0)&&(tally=="Sell")){toll ++; if(Price>signal){fg="Up";} tally="Buy";}
}
if((((Price<signal)|(|(Price<ilPass)|(|(price<lPass)))&&(((jC==Cc)&&(Price<LL[min-(y+1)]))|(|(iC==Cc)&&(Price<=HH[min-(y+1)]&&(((open<=Sale)|(|(Price<=Sale))|(|(open<=iSale)|(|(Price<=iSale))))))
{Alert("Sell: ",price);
if((toll==0)&&(tally=="Buy")){toll ++; if(Price<signal){fg="Down";} tally="Sell";}
}
if(((toll==1)&&((tally=="Buy")|(|fg=="Up"))))
{Alert("U+1F537 "/*, "💎"/*, "Diamond ",fg);
if(((prime==0)|(|(dime==0))&&(fg=="Up"))
{dime=0; mem=0;
if((((A==true)|(|(B==false))&&((u==true)|(|(v==false))))
{B(); if(C==true){P();} else{Q();} Alert("Bull"); Top(); E=price;} else{B(); Alert("Bull"); Top(); E=price;}
else if(((dime==0)&&(prime==1))&&(fg=="")&&(Price==signal))
{dime=0;
if((((A==true)|(|(B==false))&&((u==true)|(|(v==false))))
{B(); if(C==true){P();} else{Q();} Alert("Bull"); Top(); E=price;} else{B(); Alert("Bull"); Top(); E=price;}
else if(Price==signal){B(); Alert("U+1F943 "/*, "🍷"/*, "Whisky"); Top(); E=price;} else{OnLog(); A(); Alert("U+1F377 "/*, "🍷"/*, "Wine"); Bott(); D=price;}
signal=0; toll=0; tally=""; GF=true; signature=true;
}
if(((toll==1)&&((tally=="Sell")|(|fg=="Down"))))
{Alert("U+1F53B "/*, "♥"/*, "Ruby ",fg);
if((fg=="Down")&&(prime==1)|(|(dime==1)))
{dime=1; mem=1;
}
```



```
if(((A==false)||(B==true))&&((u==false)||(v==true)))
{A(); if(C==false){P();} else{Q();} Alert("Bear"); Bott(); D=price;} else{A(); Alert("Bear"); Bott(); D=price;}}
else if(((dime==1)&&(prime==0))&&(fg=="")&&(Price<signal))
{dime=1;
if(((A==false)||(B==true))&&((u==false)||(v==true)))
{A(); if(C==false){P();} else{Q();} Alert("Bear"); Bott(); D=price;} else{A(); Alert("Bear"); Bott(); D=price;}}
else if(Price<signal){A(); Alert("U+1F377 ",/*"💎",*/"Wine"); Bott(); D=price;} else{OnLog(); B(); Alert("U+1F943 ",/*"🍷",*/"Whisky"); Top(); E=price;}
signal=0; toll=0; tally=""; GF=true; signature=true;
}
if((count==1)&&(tally=="Buy")||(fg=="Up")))
{Alert("U+1F537 ",/*"💎",*/"Diamond ",fg);
if(((prime==0)||(dime==0))&&(fg=="Up"))
{dime=0; mem=0;
if(((A==true)||(B==false))&&((u==true)||(v==false)))
{B(); if(C==true){P();} else{Q();} Alert("Hawk"); Top(); E=price;} else{B(); Alert("Hawk"); Top(); E=price;}}
else if(((dime==0)&&(prime==1))&&(fg=="")&&(Price>signal))
{dime=0;
if(((A==true)||(B==false))&&((u==true)||(v==false)))
{B(); if(C==true){P();} else{Q();} Alert("Hawk"); Top(); E=price;} else{B(); Alert("Hawk"); Top(); E=price;}}
else if(Price<signal){B(); Alert("U+1F4A7 ",/*"🔴",*/"Watter"); Top(); E=price;} else{OnLog(); A(); Alert("U+1FA78 ",/*"🔴",*/"Blood"); Bott(); D=price;}
count=0; tally=""; GF=true; signature=true;
}
}
if((count==1)&&(tally=="Sell")||(fg=="Down")))
{Alert("U+1F53B ",/*"💎",*/"Ruby ",fg);
if((fg=="Down")&&(prime==1)||(dime==1))
{dime=1; mem=1;
if(((A==false)||(B==true))&&((u==false)||(v==true)))
{A(); if(C==false){P();} else{Q();} Alert("Dove"); Bott(); D=price;} else{A(); Alert("Dove"); Bott(); D=price;}}
else if(((dime==1)&&(prime==0))&&(fg=="")&&(Price<signal))
{dime=1;
if(((A==false)||(B==true))&&((u==false)||(v==true)))
{A(); if(C==false){P();} else{Q();} Alert("Dove"); Bott(); D=price;} else{A(); Alert("Dove"); Bott(); D=price;}}
else if(Price<signal){A(); Alert("U+1FA78 ",/*"🔴",*/"Blood"); Bott(); D=price;} else{OnLog(); B(); Alert("U+1F4A7 ",/*"🔴",*/"Watter"); Top(); E=price;}
count=0; tally=""; GF=true; signature=true;
}
}
}
}
if((OnGaurd(-1))&&(KC==true))
{
if((h!=0)&&(ab==false)&&(U[0-(y+1)]=true)&&(0>2)&&(0!=x-1)/*&&(OnFire(0,"sTrend","tTrend"))*/)
{
if(HH[0-(y+1)]>Premium[0-(y+1)])
{
h=0;
if((C==true)|| (c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"0:",0,"|",C,":",c);
}
}
else if(((C==false)&&(c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"0:",0,"|",C,":",c);
}
}
}
if(LL[0-(y+1)]<Discount[0-(y+1)])
{
h=0;
if((C==true)|| (c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"0:",0,"|",C,":",c);
}
}
else if(((C==false)&&(c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"0:",0,"|",C,":",c);
}
}
}
}
if((h!=0)&&(ab==false)&&(U[o-(y+1)]=true)&&(o>2)&&(o!=x-1)/*&&(OnFire(o,"sTrend","tTrend"))*/)
{
if(HH[o-(y+1)]>Premium[o-(y+1)])
{
h=o;
if((C==false)&&(c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"o:",o,"|",C,":",c);
}
}
else if(((C==true)|| (c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"o:",o,"|",C,":",c);
}
}
}
if(LL[o-(y+1)]<Discount[o-(y+1)])
{
h=o;
if((C==false)&&(c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"o:",o,"|",C,":",c);
}
}
else if(((C==true)|| (c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"o:",o,"|",C,":",c);
}
}
}
}
}
}
else if(OnGaurd(-1)!=KC)
{
if((h!=0)&&(ab==false)&&(U[0-(y+1)]=true)&&(0>2)&&(0!=x-1)/*&&(OnFire(0,"sTrend","tTrend"))*/)
{
if(HH[0-(y+1)]>Premium[0-(y+1)])
{
h=0;
if((C==false)|| (c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"0:",0,"|",C,":",c);
}
}
else if(((C==true)&&(c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"0:",0,"|",C,":",c);
}
}
}
}
if(LL[0-(y+1)]<Discount[0-(y+1)])
{
h=0;
if((C==false)|| (c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"0:",0,"|",C,":",c);
}
}
else if(((C==true)&&(c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"0:",0,"|",C,":",c);
}
}
}
}
}
if((h!=0)&&(ab==false)&&(U[o-(y+1)]=true)&&(o>2)&&(o!=x-1)/*&&(OnFire(o,"sTrend","tTrend"))*/)
{
if(HH[o-(y+1)]>Premium[o-(y+1)])
{
h=o;
if((C==true)&&(c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"o:",o,"|",C,":",c);
}
}
else if(((C==false)|| (c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"o:",o,"|",C,":",c);
}
}
}
}
if(LL[o-(y+1)]<Discount[o-(y+1)])
{
h=o;
if((C==true)&&(c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price,"o:",o,"|",C,":",c);
}
}
else if(((C==false)|| (c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price,"o:",o,"|",C,":",c);
}
}
}
}
}
}
}
if((h!=0)&&(signal!=0)&&(ab==ba))
{
if((OnGaurd(-1))&&(KC==true))
{
if(((iz>h)&&(iz>2)&&(((iz>2)&&((iz==iz)|| (iz==iz+h)|| ((iz==iz+io)&&(l[io-(y+1)]=false)/*&&(OnHold(o,"sRange","tRange"))*/))))||((I>2)&&(I==iz)|| (I==iz+h)|| ((I==iz+io)&&(l[io-(y+1)]=false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(k[iz-(y+1)]=false)/*&&(OnHold(z,"sTrend","tTrend"))*/)
{
h=iz;
if((C==false)&&(c==false))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"iz:",iz,"I:",I,"|=iz:",iz,"|",C);
}
}
else if(((C==true)|| (c==true))
{
G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"iz:",iz,"I:",I,"|=iz:",iz,"|",C);
}
}
}
}
}
```



```

    else if(((i0>=h)&&(i0>2)&&(((iz>2)&&(iz==i0)||iz==i0+h)||((iz==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==i0)||I==i0+h)||((I==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(k[i0-(y+1)]==false)/*&&(OnHold(0,"sRange","tRange"))*/)
    {
        h=i0;
        if((C==false)&&(c==false))
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"o:",o,"iZ:",iZ,"I:",I,"|=i0:",i0,"|",C);
        }
        if((C==true)||c==true)
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"o:",o,"iZ:",iZ,"I:",I,"|=i0:",i0,"|",C);
        }
    }
    if((iw>=h)&&(iw>2)&&(((iw>2)&&(iw==iw)||iw==iw+h)||((iw==iw+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==iw)||I==iw+h)||((I==iw+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(l[iw-(y+1)]==false)/*&&(OnHold(w,"sTrend","tTrend"))*/)
    {
        h=iw;
        if((C==false)&&(c==false))
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"iW:",iW,"I:",I,"|=iw:",iw,"|",C);
        }
        else if(((C==true)||c==true))
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"iW:",iW,"I:",I,"|=iw:",iw,"|",C);
        }
    }
    }
    else if((i0>=h)&&(i0>2)&&(((iw>2)&&(iw==i0)||iw==i0+h)||((iw==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==i0)||I==i0+h)||((I==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(l[i0-(y+1)]==false)/*&&(OnHold(0,"sRange","tRange"))*/)
    {
        h=i0;
        if((C==false)&&(c==false))
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"o:",o,"iW:",iW,"I:",I,"|=i0:",i0,"|",C);
        }
        else if(((C==true)||c==true))
        {
            G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"o:",o,"iW:",iW,"I:",I,"|=i0:",i0,"|",C);
        }
    }
    }
    else if(OnGaurd(-1)!=KC)
    {
        if(((iz>=h)&&(iz>2)&&(((iz>2)&&(iz==iz)||iz==iz+h)||((iz==iz+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==iz)||I==iz+h)||((I==iz+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(k[iz-(y+1)]==false)/*&&(OnHold(z,"sTrend","tTrend"))*/)
        {
            h=iz;
            if((C==true)&&(c==true))
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"iZ:",iZ,"I:",I,"|=iz:",iz,"|",C);
            }
            else if((C==false)||c==false)
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"iZ:",iZ,"I:",I,"|=iz:",iz,"|",C);
            }
        }
        else if((i0>=h)&&(i0>2)&&(((iz>2)&&(iz==i0)||iz==i0+h)||((iz==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==i0)||I==i0+h)||((I==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(k[i0-(y+1)]==false)/*&&(OnHold(0,"sRange","tRange"))*/)
        {
            h=i0;
            if((C==true)&&(c==true))
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"o:",o,"iZ:",iZ,"I:",I,"|=i0:",i0,"|",C);
            }
            if((C==false)||c==false)
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"o:",o,"iZ:",iZ,"I:",I,"|=i0:",i0,"|",C);
            }
        }
        if(((iw>=h)&&(iw>2)&&(((iw>2)&&(iw==iw)||iw==iw+h)||((iw==iw+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==iw)||I==iw+h)||((I==iw+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(l[iw-(y+1)]==false)/*&&(OnHold(w,"sTrend","tTrend"))*/)
        {
            h=iw;
            if((C==true)&&(c==true))
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"iW:",iW,"I:",I,"|=iw:",iw,"|",C);
            }
            else if(((C==false)||c==false))
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"iW:",iW,"I:",I,"|=iw:",iw,"|",C);
            }
        }
        else if((i0>=h)&&(i0>2)&&(((iw>2)&&(iw==i0)||iw==i0+h)||((iw==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))||(((I>2)&&(I==i0)||I==i0+h)||((I==i0+io)&&(l[i0-(y+1)]==false)/*&&(OnHold(o,"sRange","tRange"))*/))))&&(l[i0-(y+1)]==false)/*&&(OnHold(0,"sRange","tRange"))*/)
        {
            h=i0;
            if((C==true)&&(c==true))
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ dime=mem; Alert("Sin.",price," h:",h,"o:",o,"iW:",iW,"I:",I,"|=i0:",i0,"|",C);
            }
            else if((C==false)||c==false)
            {
                G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ dime=mem; Alert("Sine.",price," h:",h,"o:",o,"iW:",iW,"I:",I,"|=i0:",i0,"|",C);
            }
        }
    }
    }
    }
    tick ++;
    if(ab1=ba){ab=ba;} tickTock = false;
}
void OnGoe()
{if(/*(ab==false)&&*/(signal==0))
{
    if((OnGaurd(0))&&(KC==true))
    {
        if(((h==io)&&(z>o)||((h==i0)&&(Z>0))||((h==iz)&&(Z>z))||((h==iZ)&&(Z<z))))
        {
            if((C==false)&&(c==false))
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sign.",price," h:",h,"|", "Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
            else
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sig.",price," h:",h,"|", "Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
        }
        else if(((h==io)||h==iZ)||h==iz)||h==i0)))
        {
            if((C==false)&&(c==false))
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sig.",price," h:",h,"Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
            else
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sign.",price," h:",h,"Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
        }
    }
    else if(OnGaurd(0)!=KC)
    {
        if(((h==io)&&(z>o)||((h==i0)&&(Z>0))||((h==iz)&&(Z>z))||((h==iZ)&&(Z<z))))
        {
            if((C==false)||c==false)
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sign.",price," h:",h,"|", "Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
            else
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sig.",price," h:",h,"|", "Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
        }
        else if(((h==io)||h==iZ)||h==iz)||h==i0)))
        {
            if((C==false)||c==false)
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sig.",price," h:",h,"Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
            else
            {
                prime=1; G(); Signal(); tickTock=true; /*if(KC==true) */tag=1;/* else tag=0;*/ Alert("Sign.",price," h:",h,"Z:",iZ,"z:",iz,"0:",i0,"o:",io,"|",C,":",c);
            }
        }
    }
    }
    KC();
}
}
}
void OnToe()
{if(/*(ab==false)&&*/(signal==0))
{
    if((OnGaurd(0))&&(KC==true))
    {
        if(((h==io)&&(w>o)||((h==i0)&&(W>0))||((h==iw)&&(W>w))||((h==iW)&&(W<w))))
        {
            if((C==false)&&(c==false))
            {
                prime=0; G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ Alert("Sig.",price," h:",h," W<w","|", "W:",iW,"w:",iw,"0:",i0,"o:",io,"|",C,":",c);
            }
            else
            {
                prime=0; G(); Signal(); tickTock=true; /*if(KC==true) */tag=0;/* else tag=1;*/ Alert("Sig.",price," h:",h," W<w","|", "W:",iW,"w:",iw,"0:",i0,"o:",io,"|",C,":",c);
            }
        }
    }
}
```



```
if(GF==true){OnReInit(); GF=false;}
Comment("      ^",iZ,";",Z,"|",iz,";",z,"=",k[Z-(y+1)],"|",k[z-(y+1)],"      Up(0)|Down(1): ",dime,
"\n Lim",iO,";",0,"^",k[O-(y+1)]," _ ",l[O-(y+1)]," _ ",io,";",o,"^",k[o-(y+1)]," _ ",l[o-(y+1)],"=",h," _ ",C,";",c,
"\n _",iW,";",W,"|",iw,";",w,"=",l[W-(y+1)],"|",l[w-(y+1)],"      U+1F48E"/*, "💎 "*/);
} // Natalia Tanyatia
```